# Lab 05: Reversing and Using an Undocumented API

In this lab, you will dissect an undocumented API, produce usable documentation for it, and write a new software component for that API.

## Purpose

To practice using static and dynamic analysis tools to discover and understand undocumented APIs. You will write documentation for the API and/or provide software artifacts that could help a developer write an application for the API. You will write a substantial software component that communicates with an existing component via the undocumented API. Essentially, you will learn to observe, dissect, and reuse an existing, undocumented, software component in a new program.

## Procedure

You will need access to the following materials:

- A computer with a working, licensed copy of Linux, macOS, or Windows.
- A working, licensed development environment for C.
- A suite of reverse-engineering tools likely consisting of at least one static and one dynamic analysis tool.
- A copy of this lab's "connect four" program for your OS.

### Reverse Engineering the API

You may choose environments and tools to your preference. If you have previous reverse engineering experience, I encourage you to use a few tools you are less familiar with. As you explore, use each tool for its strengths and consider its weaknesses. For example, you may use a static analysis tool to identify interesting places to put breakpoints in your dynamic tool. You might also be stepping through code in a debugger and become curious about alternative code paths. A static analysis tool can help you understand better how the program got to where it is, and what conditions might have taken it elsewhere.

Take note of how the program is distributed. You should see two components: a dynamic library, and an executable. Your task is to dissect the interface between these two components and provide its documentation.

If you are paranoid, you may start with some basic static analysis. Check out what other libraries the components communicate with, see what functions they import and export, and convince yourself that the program interacts only with the console. Otherwise, start simply by running the program and getting a feel for how it works. This may provide some insight and help you form hypotheses about the structure of the API. At this point, you may continue with either static or dynamic analysis as you see fit.

### Documenting the API

Use your notes from the previous phase to produce formal documentation for the API. You should include this in a special section or appendix in your written report. At a high level, please describe the purpose of each module (file). At a lower level, please describe the functions and data types.

You may find it useful to produce compatible header files for the API and include doxygen-style documentation. This will also give you a head start in the following phase. Whether or not you use a documentation tool, please include a brief description of each function and data type. For every function, include the type and purpose of each parameter, and, if applicable, the type and purpose of its return value. For every non-primitve data type, include the type and purpose of each field. You may also need to describe the structure of any complicated data types. Basically, you should provide enough detail that another developer could use your notes to produce a compatible component without additional reverse engineering.

### Forward or Re-Engineering

Replace the `connect4` library so that it plays "tic tac toe" instead. Create the appropriate header file(s) to communicate via the API. To ensure your component interfaces correctly, this header will need to be accurate.

You should be able to achieve the desired effect by compiling your new module and dropping it in place of the old without modifying the driver. You *may not* use any I/O functions or system calls, except `fprintf()` for debugging. Only the driver may ask for input.

## Tips and Caveats

- You will probably want to use a debugger to break at each of the exported entry points of `connect4`.
- There is a function pointer callback mechanism. Be sure to document its signature.
- These phases need not be considered strictly separate steps. If you find your documentation to be lacking as you work to implement your new module, go back and do more RE work. Similarly, if you prefer to begin implementing parts of your replacement module as you devise your RE experiments, feel free to do so.

## Report

Completion of this lab must be substantiated by a report prepared by the group. It is up to each group to divide its responsibilities fairly. The report should provide sufficient detail that another person could verify your work.

Please include an archive of your source code and resulting binaries in your electronic submission.

## Grading

The report should describe the purpose of the lab, the procedure, any assumptions, problems, or hypotheses, and the final results. Mostly, I'll be looking for a proper problem statement, a structured procedure, and for a reasonably correct solution demonstrated by your results.

I will attempt to build your source code and run the program with your replacement module. Part of your grade will depend on its successful execution. Please only submit one replacement module. If you submit multiple, I may choose one at random.

This lab is due midnight after 2 weeks.

**Rubric**

| points | Accomplishment |
|---|---|
| 3 | Report is well-written (organized, detailed, illustrative) |
| 2 | Report provides correct API documentation |
| 1 | The replacement module compiles for the target platform |
| 1 | The program with replacement runs without error |
| 1 | The program with replacement plays "tic tac toe" |
| 2 | Only the driver interacts directly with the user |